PYTHON · INFORMATIK

Schleifen I

Die for-Schleife

Theresa Luternauer & Julia Imhof

HS 2026/27

Kantonsschule Uster

Wiederholungen mit der `for`-Schleife

Manche Anweisungen müssen mehrfach ausgeführt werden. Mit der `for`-Schleife kannst du solche Wiederholungen präzise steuern. Du lernst, wie Schleifenvariablen und `range()` funktionieren und wie du damit übersichtliche Programme schreibst.

Worum geht es?

Computer eignen sich besonders gut dazu, wiederkehrende Aufgaben automatisch auszuführen. In diesem Kapitel lernst du die `for`-Schleife kennen. Mit ihr kannst du festlegen, **wie oft** ein Codeblock ausgeführt werden soll.

Du lernst ausserdem die Funktion `range()` genauer kennen. Mit `range()` kannst du nicht nur bei 0 beginnen und in Einerschritten zählen, sondern auch einen Startwert und eine Schrittweite festlegen.

Lernziele

Nach diesem Kapitel kannst du …

- erklären, warum Wiederholungen beim Programmieren wichtig sind,
- eine `for`-Schleife lesen und in eigenen Programmen verwenden,
- die Bedeutung der Schleifenvariable erklären,
- `range(stop)` verwenden,
- `range(start, stop)` verwenden,
- `range(start, stop, schrittweite)` verwenden,
- die von `range()` erzeugten Zahlenfolgen bestimmen,
- mit einer negativen Schrittweite rückwärts zählen,
- Turtle-Zeichnungen mithilfe von Schleifen verkürzen,
- einfache verschachtelte `for`-Schleifen nachvollziehen und anwenden.

Warum brauchen wir Wiederholungen?

Wiederholungen sind ein wichtiger Bestandteil fast aller Programmiersprachen. Sie ermöglichen:

- **Automatisierung:** Der Computer führt eine Tätigkeit viele Male aus, ohne dass jede Wiederholung einzeln programmiert werden muss.
- **Kürzeren Code:** Wiederholte Anweisungen müssen nur einmal aufgeschrieben werden.
- **Bessere Lesbarkeit:** Die Struktur und die Absicht eines Programms werden deutlicher.
- **Flexibilität:** Die Anzahl der Wiederholungen kann leicht geändert werden.
- **Weniger Fehler:** Code, der nur einmal geschrieben wird, muss auch nur einmal korrigiert werden.

Beispiel: Ein Quadrat ohne Schleife

```
import turtle

t = turtle.Turtle()

t.forward(50)
t.left(90)
t.forward(50)
t.left(90)
```

```
t.forward(50)
t.left(90)
t.forward(50)
t.left(90)
```

Die beiden Anweisungen

```
t.forward(50)
t.left(90)
```

werden viermal wiederholt. Das Programm funktioniert, enthält aber unnötig viel gleichartigen Code.

Mit einer `for`-Schleife kann dieselbe Zeichnung kürzer beschrieben werden:

```
import turtle

t = turtle.Turtle()

for i in range(4):
    t.forward(50)
    t.left(90)
```

!!! tip “Merke” Wenn derselbe Code mehrmals hintereinander ausgeführt werden soll, ist häufig eine Schleife sinnvoll.

Aufbau einer `for`-Schleife

Eine `for`-Schleife hat folgenden Aufbau:

```
for schleifenvariable in range(anzahl):
    anweisung_1
    anweisung_2
```

Beispiel:

```
print("Start")

for i in range(3):
    print("Hallo")

print("Stopp")
```

Ausgabe:

```
Start
Hallo
Hallo
Hallo
Stopp
```

Die Zeile

```
for i in range(3):
```

heisst **Schleifenkopf**. Die eingerückten Anweisungen bilden den **Schleifenkörper**.

Der Schleifenkörper wird in diesem Beispiel dreimal ausgeführt.

Die Einrückung

Nur die eingerückten Anweisungen gehören zur Schleife:

```
for i in range(3):
    print("wird wiederholt")

print("wird nur einmal ausgeführt")
```

> **Achtung:** Nach dem Doppelpunkt müssen alle Anweisungen eingerückt werden, die wiederholt werden sollen. Eine nicht mehr eingerückte Anweisung gehört nicht zur Schleife.

Die Schleifenvariable

Im folgenden Beispiel ist `i` die **Schleifenvariable**:

```
for i in range(4):
    print(i)
```

Ausgabe:

```
0
1
2
3
```

Die Schleifenvariable erhält in jedem Schleifendurchgang den nächsten Wert aus `range(4)`.

Schleifendurchgang	Wert von `i`
1. Durchgang	0
2. Durchgang	1
3. Durchgang	2
4. Durchgang	3

> **Merke: “Zählen ab 0”**
> `range(4)` erzeugt die Werte 0, 1, 2 und 3. Die Schleife hat vier Durchgänge, obwohl der letzte Wert der Schleifenvariable 3 ist.

Der Name der Schleifenvariable kann selbst gewählt werden. Häufig werden kurze Namen wie `i` verwendet:

```
for runde in range(4):
    print(runde)
```

Ist die Bedeutung wichtig, ist ein beschreibender Name wie `runde` oft leichter verständlich.

Die Schleifenvariable verwenden

Die Schleifenvariable kann im Schleifenkörper verwendet werden:

```python
for i in range(5):
    print("Runde", i)
```

Ausgabe:

```
Runde 0
Runde 1
Runde 2
Runde 3
Runde 4
```

Wenn der aktuelle Wert nicht benötigt wird, kann als Variablenname `_` verwendet werden:

```python
for _ in range(4):
    print("Hallo")
```

Das `_` zeigt: Die Schleife soll viermal laufen, aber der aktuelle Zahlenwert wird im Schleifenkörper nicht gebraucht.

Funktion `range()`

Die Funktion `range()` liefert eine Folge ganzer Zahlen. Eine `for`-Schleife durchläuft diese Zahlen der Reihe nach.

`range()` kann in drei Formen verwendet werden:

```python
range(stop)
range(start, stop)
range(start, stop, schrittweite)
```

> **Achtung: "Der Endwert ist nicht enthalten"** Der Wert `stop` gehört nie zur erzeugten Zahlenfolge. `range()` hört unmittelbar vor dem Endwert auf.

`range(stop)`

Wird nur ein Wert angegeben, beginnt die Zahlenfolge bei 0 und zählt in Einerschritten:

```python
range(stop)
```

Beispiele:

Ausdruck	Erzeugte Werte	Anzahl Werte
`range(3)`	0, 1, 2	3
`range(5)`	0, 1, 2, 3, 4	5
`range(1)`	0	1
`range(0)`	keine Werte	0

Die allgemeine Regel lautet:

```
0, 1, 2, ..., stop - 1
```

Beispiel:

```
for i in range(5):
    print(i)
```

Ausgabe:

```
0
1
2
3
4
```

`range(start, stop)`
Mit zwei Werten wird zusätzlich der Startwert festgelegt:

```
range(start, stop)
```

Die Zahlenfolge beginnt bei `start` und zählt in Einerschritten. Der Endwert `stop` ist wiederum nicht enthalten.

Ausdruck	Erzeugte Werte
`range(2, 6)`	2, 3, 4, 5
`range(5, 10)`	5, 6, 7, 8, 9
`range(7, 8)`	7
`range(4, 4)`	keine Werte

Beispiel:

```
for i in range(3, 7):
    print(i)
```

Ausgabe:

```
3
4
5
6
```

`range(start, stop, schrittweite)`
Mit drei Werten wird auch die Schrittweite festgelegt:

```
range(start, stop, schrittweite)
```

Die Schrittweite gibt an, um wie viel sich der Wert nach jedem Durchgang verändert.

Ausdruck	Erzeugte Werte
`range(2, 10, 2)`	2, 4, 6, 8
`range(1, 10, 3)`	1, 4, 7
`range(5, 16, 3)`	5, 8, 11, 14
`range(0, 21, 5)`	0, 5, 10, 15, 20

Auch hier ist `stop` nicht enthalten. Es spielt keine Rolle, ob der Endwert mit der Schrittweite genau erreicht werden könnte: Sobald der nächste Wert den Endwert erreichen oder überschreiten würde, endet die Zahlenfolge.

Beispiel:

```
for i in range(5, 16, 3):
    print(i)
```

Ausgabe:

```
5
8
11
14
```

Rückwärts zählen

Für eine rückwärts laufende Zahlenfolge wird eine negative Schrittweite verwendet:

```
range(start, stop, negative_schrittweite)
```

Beispiele:

Ausdruck	Erzeugte Werte
`range(5, 0, -1)`	5, 4, 3, 2, 1
`range(10, 0, -2)`	10, 8, 6, 4, 2
`range(12, 3, -3)`	12, 9, 6

Beispiel für einen Countdown:

```
for i in range(5, 0, -1):
    print(i)

print("Start!")
```

Ausgabe:

```
5
4
3
2
1
Start!
```

Achtung: "Richtung und Schrittweite müssen zusammenpassen" Soll von einer grösseren zu einer kleineren Zahl gezählt werden, muss die Schrittweite negativ sein. `range(5, 0)` liefert keine Werte, weil die Standardschrittweite `+1` vom Endwert wegführt.

Die drei Formen im Vergleich

Form	Startwert	Endwert	Schrittweite
`range(stop)`	0	nicht enthalten	1
`range(start, stop)`	angegeben	nicht enthalten	1
`range(start, stop, schrittweite)`	angegeben	nicht enthalten	angegeben

Tipp: "So bestimmst du eine `range()` -Folge"

1. Schreibe den Startwert auf.
2. Addiere jeweils die Schrittweite.
3. Stoppe, bevor der Endwert erreicht oder überschritten wird.
4. Bei einer negativen Schrittweite stoppst du, bevor der Endwert erreicht oder unterschritten wird.

Übungen zu `range()`

Aufgabe 1: Zahlenfolgen bestimmen

Notiere die Werte, die `range()` jeweils erzeugt.

a) `range(4)`

b) `range(7)`

c) `range(1)`

d) `range(0)`

Aufgabe 2: Mit Start- und Endwert

Notiere die erzeugten Werte.

a) `range(2, 6)`

b) `range(5, 9)`

c) `range(8, 11)`

d) `range(4, 4)`

Aufgabe 3: Mit Schrittweite

Notiere die erzeugten Werte.

a) `range(0, 10, 2)`

b) `range(1, 12, 3)`

c) `range(5, 20, 5)`

d) `range(4, 15, 4)`

Aufgabe 4: Rückwärts zählen
Notiere die erzeugten Werte.

a) `range(5, 0, -1)`

b) `range(10, 4, -2)`

c) `range(20, 0, -5)`

d) `range(3, 8, -1)`

Aufgabe 5: Anzahl Durchgänge
Bestimme, wie oft der Schleifenkörper ausgeführt wird.

a)

```
for i in range(6):
    print(i)
```

b)

```
for i in range(3, 8):
    print(i)
```

c)

```
for i in range(0, 12, 3):
    print(i)
```

d)

```
for i in range(10, 0, -2):
    print(i)
```

Aufgabe 6: Passenden `range()`-Ausdruck finden
Gib jeweils einen passenden `range()`-Ausdruck an.

a) 0, 1, 2, 3, 4
b) 3, 4, 5, 6, 7
c) 2, 4, 6, 8, 10
d) 10, 20, 30, 40, 50
e) 5, 4, 3, 2, 1
f) 20, 15, 10, 5

Aufgabe 7: Fehler erklären
Eine Schülerin möchte von 5 bis 1 rückwärts zählen und schreibt:

```
for i in range(5, 1):
    print(i)
```

a) Weshalb erscheint keine Ausgabe?
b) Wie lautet der korrekte Schleifenkopf?
c) Welcher Endwert muss verwendet werden, damit auch die Zahl 1 ausgegeben wird?

Aufgabe 8: Endwert verstehen
Beantworte ohne Computer.

a) Weshalb enthält `range(1, 5)` die Zahl 5 nicht?

b) Wie muss `range()` geschrieben werden, damit die Zahlen 1 bis und mit 5 erzeugt werden?

c) Wie muss `range()` geschrieben werden, damit alle geraden Zahlen von 2 bis und mit 20 erzeugt werden?

Programme mit `for` -Schleifen lesen

Beispiel 1: Text wiederholen

```
for i in range(3):
    print("Python")
```

Der Schleifenkörper wird dreimal ausgeführt. Das Wort `Python` erscheint dreimal.

Beispiel 2: Schleifenvariable ausgeben

```
for i in range(2, 6):
    print("i hat den Wert", i)
```

Ausgabe:

```
i hat den Wert 2
i hat den Wert 3
i hat den Wert 4
i hat den Wert 5
```

Beispiel 3: Mit der Schleifenvariable rechnen

```
for i in range(1, 6):
    print(i, "mal 3 ist", i * 3)
```

Ausgabe:

```
1 mal 3 ist 3
2 mal 3 ist 6
3 mal 3 ist 9
4 mal 3 ist 12
5 mal 3 ist 15
```

Aufgaben

Aufgabe 9: Ausgabe vorhersagen

Notiere die vollständige Ausgabe.

```
print("A")

for i in range(3):
    print(i)

print("B")
```

Aufgabe 10: Ausgabe vorhersagen

```
for zahl in range(2, 9, 2):
    print(zahl * 10)
```

a) Welche Werte erhält `zahl` ?
b) Welche Zahlen werden ausgegeben?
c) Wie oft wird die `print()` -Anweisung ausgeführt?

Aufgabe 11: Programm ergänzen

Ergänze nur den `range()` -Ausdruck.

a) Gib die Zahlen 1 bis 10 aus.

```
for i in range(_________________):
    print(i)
```

b) Gib die ungeraden Zahlen von 1 bis 19 aus.

```
for i in range(_________________):
    print(i)
```

c) Gib die Zahlen 10, 8, 6, 4 und 2 aus.

```
for i in range(_________________):
    print(i)
```

Turtle-Zeichnungen mit Schleifen

Bei Turtle-Zeichnungen wiederholen sich häufig dieselben Bewegungen. Schleifen helfen dabei, geometrische Figuren kurz und übersichtlich zu programmieren.

Quadrat

Ein Quadrat besteht aus vier gleich langen Seiten und vier Drehungen um 90 Grad:

```
import turtle

t = turtle.Turtle()

for i in range(4):
    t.forward(80)
    t.left(90)
```

Der gesamte Schleifenkörper muss viermal ausgeführt werden. Deshalb sind beide Anweisungen eingerückt.

Gleichseitiges Dreieck

```
for i in range(3):
    t.forward(80)
    t.left(120)
```

Regelmässige Vielecke

Bei einem regelmässigen Vieleck gilt für den Drehwinkel:

```
Drehwinkel = 360 / Anzahl Seiten
```

Ein regelmässiges Sechseck:

```
for i in range(6):
    t.forward(60)
    t.left(360 / 6)
```

Ein regelmässiges Zwölfeck:

```
for i in range(12):
    t.forward(30)
    t.left(360 / 12)
```

!!! note “Zusammenhang” Die Anzahl der Schleifendurchgänge entspricht der Anzahl Seiten. Nach jeder Seite dreht sich die Turtle um `360 / anzahl_seiten` Grad.

Die Schleifenvariable für Veränderungen verwenden

Die Schleifenvariable kann auch die Zeichnung beeinflussen:

```
for i in range(10, 110, 10):
    t.forward(i)
    t.left(90)
```

Die Werte von `i` lauten:

```
10, 20, 30, 40, 50, 60, 70, 80, 90, 100
```

Die Linien werden deshalb immer länger. Es entsteht eine eckige Spirale.

Aufgabe 12: Figuren mit Schleifen

Zeichne mit einer `for`-Schleife:

a) ein Quadrat,
b) ein gleichseitiges Dreieck,
c) ein regelmässiges Fünfeck,
d) ein regelmässiges Achteck.

Aufgabe 13: Fehler finden

Das folgende Programm soll ein Quadrat zeichnen:

```
for i in range(4):
    t.forward(50)
t.left(90)
```

a) Weshalb entsteht kein Quadrat?
b) Korrigiere die Einrückung.
c) Erkläre, wie oft `t.left(90)` im fehlerhaften Programm ausgeführt wird.

Verschachtelte `for`-Schleifen

Manchmal wird eine Wiederholung innerhalb einer anderen Wiederholung benötigt. Dann verwenden wir eine **verschachtelte Schleife**.

```
for i in range(3):
    for j in range(2):
        print("i =", i, "j =", j)
```

Ausgabe:

```
i = 0 j = 0
i = 0 j = 1
i = 1 j = 0
i = 1 j = 1
i = 2 j = 0
i = 2 j = 1
```

Für jeden einzelnen Wert der äusseren Schleifenvariable `i` wird die innere Schleife mit `j` vollständig ausgeführt.

Wert von `i`	Werte von `j`
0	0, 1
1	0, 1
2	0, 1

Die äussere Schleife hat 3 Durchgänge. Die innere Schleife hat bei jedem äusseren Durchgang 2 Durchgänge. Insgesamt wird die `print()`-Anweisung daher

```
3 · 2 = 6 Mal
```

ausgeführt.

Analogie: Uhr

Bei einer Uhr bewegt sich der Stundenzeiger langsam. Für jede Stunde durchläuft der Minutenzeiger einmal alle Minuten. Ähnlich läuft bei verschachtelten Schleifen die innere Schleife bei jedem Durchgang der äusseren Schleife vollständig ab.

Verschachtelte Turtle-Schleife

Das folgende Programm zeichnet zwölf Quadrate. Nach jedem Quadrat wird die Turtle um 30 Grad gedreht:

```
for i in range(12):
    for j in range(4):
        t.forward(60)
        t.left(90)

    t.left(30)
```

- Die **innere Schleife** zeichnet ein Quadrat.
- Die **äussere Schleife** wiederholt das Quadrat zwölfmal.
- Nach jedem Quadrat dreht sich die Turtle um 30 Grad.

Die Einrückung zeigt, dass `t.left(30)` zur äusseren Schleife gehört, aber nicht zur inneren Schleife.

Aufgabe 14: Verschachtelte Schleife lesen

```
for i in range(2):
    for j in range(3):
        print(i, j)
```

a) Welche Werte erhält `i` ?

b) Welche Werte erhält `j` ?

c) Notiere die vollständige Ausgabe.

d) Wie oft wird `print(i, j)` insgesamt ausgeführt?

Aufgabe 14: Anzahl Wiederholungen bestimmen

Wie oft wird jeweils die eingerückte Anweisung ausgeführt?

a)

```
for i in range(4):
    for j in range(5):
        print("Hallo")
```

b)

```
for i in range(2, 6):
    for j in range(0, 10, 2):
        print(i, j)
```

Aufgabe 15: Kleines Einmaleins

Ergänze das Programm so, dass das kleine Einmaleins von 1 bis 10 ausgegeben wird:

```
for zeile in range(_________________):
    for spalte in range(_________________):
        print(zeile * spalte, end=" ")

    print()
```

Erkläre danach:

a) Welche Aufgabe hat die äussere Schleife?

b) Welche Aufgabe hat die innere Schleife?

c) Weshalb wird am Ende jeder Zeile ein leeres `print()` benötigt?

Typische Fehler

Fehler 1: Doppelpunkt vergessen

```
for i in range(4)
    print(i)
```

Korrekt:

```
for i in range(4):
    print(i)
```

Fehler 2: Schleifenkörper nicht eingerückt

```
for i in range(4):
print(i)
```

Korrekt:

```
for i in range(4):
    print(i)
```

Fehler 3: Endwert wird mitgezählt

```
range(1, 5)
```

liefert nicht 1, 2, 3, 4, 5, sondern:

```
1, 2, 3, 4
```

Soll auch 5 enthalten sein, muss der Endwert 6 verwendet werden:

```
range(1, 6)
```

Fehler 4: Falsche Einrückung bei verschachtelten Schleifen

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

Jede zusätzliche Verschachtelung benötigt eine weitere Einrückung.

Zusammenfassung

!!! summary “Das Wichtigste in Kürze” - Eine `for`-Schleife wiederholt einen eingerückten Codeblock. - Die Schleifenvariable erhält nacheinander die Werte aus `range()`. - `range(stop)` beginnt bei 0 und endet vor `stop`. - `range(start, stop)` beginnt bei `start` und endet vor `stop`. - `range(start, stop, schrittweite)` verändert den Wert jeweils um die angegebene Schrittweite. - Der Endwert `stop` ist nie enthalten. - Für das Rückwärtszählen wird eine negative Schrittweite benötigt. - Die Einrückung bestimmt, welche Anweisungen wiederholt werden. - Verschachtelte Schleifen führen eine Wiederholung innerhalb einer anderen Wiederholung aus. - Turtle-Zeichnungen lassen sich mit Schleifen kurz, übersichtlich und flexibel programmieren.

Begriffe

Begriff	Bedeutung
Wiederholung	Mehrmalige Ausführung eines Programmteils
Schleife	Programmstruktur zur Steuerung von Wiederholungen
Schleifenkopf	Erste Zeile der Schleife mit `for`, Schleifenvariable und `range()`
Schleifenkörper	Eingerückter Codeblock, der wiederholt wird
Schleifenvariable	Variable, die nacheinander die Werte aus `range()` annimmt
`range()`	Python-Funktion, die eine Folge ganzer Zahlen bereitstellt
Startwert	Erster Wert einer `range()`-Folge
Endwert	Grenze der Folge; dieser Wert ist nicht enthalten
Schrittweite	Veränderung des Werts von einem Durchgang zum nächsten
Verschachtelte Schleife	Schleife, die sich innerhalb einer anderen Schleife befindet

---

Programmieraufgaben

Unter folgendem Link sind die Programmieraufgaben in CodeExpert zu finden:

Schleifen_Basic

Schleifen_Advanced